Лабораторна робота №8

«Форми введення даних та навігація між екранами»

Мета

Сформувати практичні навички створення форм введення даних у Flutter, навчитися використовувати віджети TextField, TextFormField, ElevatedButton, Form, TextEditingController, реалізовувати базову валідацію введених даних, а також виконувати навігацію між екранами за допомогою Navigator.

Теоретичні відомості

1. Форми введення даних у Flutter

Форма — це набір елементів інтерфейсу, призначених для введення та обробки даних користувача.

У Flutter для створення форм найчастіше використовуються:

- TextField — просте поле введення;
  TextFormField — поле введення з підтримкою валідації;
- Form — контейнер для об’єднання кількох полів форми;
- TextEditingController — об’єкт для керування текстом у полі введення;
- ElevatedButton — кнопка для підтвердження дії.

Форми використовуються для:

- входу в застосунок;
- реєстрації;
- введення контактних даних;
- пошуку;
- надсилання повідомлень.

2. Віджет TextField

TextField — це базовий віджет для введення тексту користувачем.

Його використовують, коли потрібно отримати текстове значення без складної валідації.

Приклад

```
TextField(
 decoration: InputDecoration(
   labelText: 'Введіть логін',
   border: OutlineInputBorder(),
 ),
)
```

Основні властивості TextField:

- decoration — оформлення поля;
- controller — керування текстом;
- keyboardType — тип клавіатури;
- obscureText — приховування символів для пароля;
- onChanged — обробка зміни тексту.

3. Віджет TextFormField

TextFormField — це розширений варіант TextField, який підтримує перевірку введених даних.

Його доцільно використовувати у формах входу, реєстрації або заповнення даних користувача.

Приклад

```
TextFormField(
 decoration: const InputDecoration(
   labelText: 'Email',
   border: OutlineInputBorder(),
```

```
 ),
 validator: (value) {
   if (value == null || value.isEmpty) {
     return 'Поле не може бути порожнім';
   }
   return null;
 },
)
```

Перевага TextFormField полягає в тому, що перевірку можна виконувати централізовано через Form.

4.Віджет Form

Form — це контейнер, який об’єднує кілька полів введення в одну форму.

Для керування станом форми використовується ключ:

```
final _formKey = GlobalKey<FormState>();

Form(
 key: _formKey,
 child: Column(
   children: [
     TextFormField(
       decoration: const InputDecoration(
         labelText: 'Логін',
       ),
     ),
     TextFormField(
       decoration: const InputDecoration(
         labelText: 'Пароль',
       ),
     ),
   ],
 ),
)
```

Форма дозволяє:

- перевіряти всі поля одразу;
- зручно організовувати введення даних;
- викликати validate() перед переходом на інший екран.

5. TextEditingController

TextEditingController використовується для зчитування, зміни та очищення тексту в полі введення.

Приклад

```
final loginController = TextEditingController();
final passwordController = TextEditingController();
```

Приклад контролера до поля:

```
TextField(
 controller: loginController,
 decoration: const InputDecoration(
   labelText: 'Логін',
 ),
)
```

Отримання значення:

```
String login = loginController.text;
```

Очищення поля:

```
loginController.clear();
```

Після завершення роботи контролери слід звільняти в методі dispose().

```
@override
void dispose() {
 loginController.dispose();
 passwordController.dispose();
 super.dispose();
}
```

6. Валідація введених даних

Валідація — це перевірка правильності введених користувачем даних.

Найчастіше перевіряють:

- чи поле не порожнє;
- мінімальну довжину тексту;
- правильність формату email;
- відповідність пароля умовам.

Приклад перевірки логіна

```
TextFormField(
 decoration: const InputDecoration(
   labelText: 'Логін',
 ),
 validator: (value) {
   if (value == null || value.isEmpty) {
     return 'Введіть логін';
   }
   if (value.length < 3) {
     return 'Логін повинен містити щонайменше 3 символи';
   }
   return null;
 },
)
```

Приклад перевірки пароля

```
TextFormField(
 obscureText: true,
 decoration: const InputDecoration(
   labelText: 'Пароль',
 ),
 validator: (value) {
   if (value == null || value.isEmpty) {
     return 'Введіть пароль';
   }
   if (value.length < 6) {
```

```
      return 'Пароль повинен містити щонайменше 6 символів';
    }
    return null;
  },
)
```

Перевірка форми перед виконанням дії:

```
if (_formKey.currentState!.validate()) {
// форма заповнена правильно
}
```

7. Навігація між екранами у Flutter

Навігація у Flutter дозволяє переходити між різними екранами застосунку.

Для цього найчастіше використовують клас Navigator.

Основні методи:

- Navigator.push() — перехід на новий екран;
- Navigator.pop() — повернення назад;
- Navigator.pushReplacement() — заміна поточного екрана новим.

8. Перехід з RegisterScreen на Home

Після успішного заповнення форми можна перейти з екрана входу (RegisterScreen) на головний екран (HomeScreen).

Приклад переходу

```
Navigator.push(
context,
MaterialPageRoute(
builder: (context) => const HomeScreen(),
),
);
```

Якщо потрібно замінити екран входу екраном Home, використовують:

```
Navigator.pushReplacement(
context,
MaterialPageRoute(
builder: (context) => const HomeScreen(),
),
);
```

У такому випадку користувач не повернеться на екран входу кнопкою «назад».

9. Логіка кнопки входу

Після натискання на кнопку входу зазвичай виконується така послідовність дій:

1. перевірка форми через validate();
2. зчитування логіна та пароля;
3. за потреби — виведення повідомлення;
4. перехід на екран Home.

Приклад

```
onPressed: () {
 if (_formKey.currentState!.validate()) {
   Navigator.pushReplacement(
     context,
     MaterialPageRoute(
       builder: (context) => HomeScreen(
         username: loginController.text,
       ),
     ),
   );
 }
}
```

10. Передавання даних між екранами

Під час переходу на інший екран можна передати введені дані, наприклад логін користувача.

Приклад конструктора HomeScreen

```
class HomeScreen extends StatelessWidget {
 final String username;

 const HomeScreen({
   super.key,
   required this.username,
 });

 @override
 Widget build(BuildContext context) {
   return Scaffold(
     appBar: AppBar(
       title: const Text('Home'),
     ),
     body: Center(
       child: Text('Вітаємо, $username'),
     ),
   );
 }
}
```

Приклад переходу з передаванням даних

```
Navigator.pushReplacement(
context,
MaterialPageRoute(
builder: (context) => HomeScreen(
username: loginController.text,
),
),
);
```

Завдання на лабораторну роботу

1. Створити екран реєстрації RegisterScreen.
2. Реалізувати форму введення даних за допомогою Form.
3. Додати щонайменше такі поля введення:
    - ім’я користувача;
    - дата народження;
    - email;
    - пароль;
    - підтвердження пароля.
4. Для полів введення використати TextFormField.
5. Реалізувати перевірку правильності введених даних:
    - усі поля не повинні бути порожніми;
    - ім’я користувача повинно містити не менше 2–3 символів;
    - дата народження має мати перевірку на дату;
    - email повинен містити символ @;
    - пароль повинен містити не менше 6 символів;
    - поле підтвердження пароля повинно збігатися з паролем.
6. Використати TextEditingController для зчитування введених значень.
7. Додати кнопку «Зареєструватися».
8. Реалізувати навігацію з RegisterScreen на HomeScreen.
9. Передати на екран HomeScreen ім’я користувача.
10. Відобразити на екрані HomeScreen текст привітання, наприклад: «Вітаємо, Ім’я!».
11. Використати Navigator.push() або Navigator.pushReplacement() для переходу між екранами.

12. Оформити інтерфейс екранів із використанням Scaffold, AppBar, Padding, Column.
13. Перевірити, що перехід на HomeScreen виконується лише після успішної валідації форми.

Контрольні запитання

1. Для чого у Flutter використовується віджет Form?
2. У чому різниця між TextField і TextFormField?
3. Для чого використовується TextEditingController?
4. Яке призначення методу validate()?
5. Для чого потрібен GlobalKey<FormState>?
6. Як реалізувати перевірку порожнього поля у формі?
7. Для чого використовується властивість obscureText?
8. Який віджет найчастіше використовується для кнопки входу?
9. Яке призначення Navigator.push()?
10. У чому різниця між Navigator.push() і Navigator.pushReplacement()?
11. Як передати дані з одного екрана на інший?
12. Що станеться, якщо не викликати dispose() для контролерів?